RELEASE TEST · FOR THE TESTING TEAM

Did the release actually work? Prove it with the real logs.

Release Test takes the APIs you're testing, helps you pull their log lines, and reads them back *end-to-end* — the front-end call and every back-end it triggers — to give each API a clear **pass** / **fail** and the release a single readiness verdict.

1 The workflow — four steps

1

Select APIs

Pick the APIs in scope — one, several, or all. Optionally tick changed routes/backends to pull in the impacted ones.

2

Get the logs

Copy the generated **Splunk query** and run it, then paste the Job SID to download results — *or* upload a raw **output log**.

3

Upload under "Verify with logs"

Drop in the Splunk export or output log. The shape is detected automatically.

4

Read the verdict

A readiness banner, a status donut, and every API with pass/fail, latency, attempts and its back-ends.

> The query is **scoped to the client version you're testing** and pairs each back-end with its expected **service version**, so you fetch only the right lines — less noise, faster to read.

2 What it checks — end to end

For each API it correlates the **front-end** request with the **back-end** calls it triggers, and checks:

- **Front-end result** — did the controller respond successfully?
- **Each back-end** — did every downstream call succeed? (grouped by response code when they didn't)
- **Service version** — did the back-end log the **expected** version? A mismatch (`svc 9.9 ✗ (exp 2.2)`) flags the row even if the call otherwise passed.
- **Attempts** — how many times it ran and how many passed (`3 (2✓/1✗)`) — so a flaky or retried flow is visible.

3 How the verdict is decided — coverage first, then the pass bar

Each API gets one end-to-end verdict, in this order:

Was every impacted flow actually tested?

No → **Partial** a flow has no log lines — a **coverage gap**, run the missing ones.

↓ yes, all tested

Did any flow fail?

Yes → **Failed** a back-end error or a service-version mismatch.

↓ no failures

Is the pass rate at or above the bar (default 95%)?

Yes → **Success** No → **Failed** the threshold is configurable in ⚙ Config.

> **Why coverage first:** a green pass rate on *half* the flows isn't a pass. Nothing reads **Success** until every impacted flow has been exercised — so "Partial" is your cue to run more, not to sign off.

4 What each status means

- **Success** — all flows tested and passing at/above the bar. Ready.
- **Failed** — a flow failed, or the pass rate is below the bar. Investigate.
- **Partial** — a flow wasn't tested. Coverage gap — run the missing flows, then re-check.
- **Skipped** — a back-end a rule says to ignore. Neutral; never fails the API.
- **Not tested** — no log lines matched this API at all. Nothing to conclude yet.

> **Status vs Latest Result:** "Status" is the overall end-to-end verdict above; "Latest Result" is just the most recent single call's reply. They can differ — trust **Status** for sign-off.

5 Host response-code rules (🚦 Rules)

Some back-ends report their outcome under a non-standard key (e.g. `"resultCode":"000000"`), with a custom success value, or shouldn't count at all. Open 🚦 `Rules` and, per host, tell the analyzer:

- which JSON key holds the response code,
- which values count as **success**,
- or **skip** the back-end entirely → shown as a neutral `Skipped` that never fails the API.

Save once — the rules are remembered (and can ship pre-loaded with the app), and the custom code is even folded into the generated Splunk query so your export carries it.

6 What you get

- A **readiness banner** — **Ready** / **Needs review** / **At risk** — with the front-end pass rate.
- A **status donut** and clickable filter chips (All / Issues / per-status), **worst-first** so failures surface.
- Per-API front-end latency, attempts (`n✓/n✗`), and an expandable **back-end breakdown** with the failed-response codes.
- A **PDF verification report** for the test evidence pack — per module, covering every API (even the not-tested ones).

Verdict order = coverage first (any flow untested → Partial · any flow failed → Failed), then the front-end pass rate vs the threshold (default 95%) → Success or Failed · Skipped back-ends are neutral · a service-version mismatch fails a back-end even on an otherwise-good response.